Erkennung ungenutzter Bibliotheken in Deployments zur risikobasierten Priorisierung von Sicherheitsbefunden

Projektarbeit 1 (T4_1000)

Im Rahmen der Prüfung:

Bachelor of Science (B. Sc.)

des Studienganges Informatik

an der
Dualen Hochschule Baden-Württemberg Karlsruhe

von

Abgabedatum	05.01.2026
Bearbeitungszeitraum	20.10.2025 - 05.01.2026
Matrikelnummer, Kurs	, TINF25B6
Ausbildungsfirma	SAP SE Dietmar-Hopp-Allee 16 69190 Walldorf, Deutschland
Betreuer der Ausbildungsfirma	
Gutachter der Dualen Hochschule	Prof. Nuo Li

Eidesstattliche Erklärung

Ich versichere hiermit, dass ich meine Projektarbeit 1 (T4_1000) mit dem Thema:

> *Erkennung ungenutzter Bibliotheken in Deployments zur risikobasierten Priorisierung von Sicherheitsbefunden*

gemäß § 5 der „Studien- und Prüfungsordnung DHBW Technik“ vom 29. September 2017 selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe. Die Arbeit wurde bisher keiner anderen Prüfungsbehörde vorgelegt und auch nicht veröffentlicht.

Ich versichere zudem, dass die eingereichte elektronische Fassung mit der gedruckten Fassung übereinstimmt.

Karlsruhe, 15. Februar 2026

gez.: ,

Sperrvermerk

Die nachfolgende Arbeit enthält vertrauliche Daten der:

SAP SE
Dietmar-Hopp-Allee 16
69190 Walldorf
Deutschland

Der Inhalt dieser Arbeit darf weder als Ganzes noch in Auszügen Personen außerhalb des Prüfungsprozesses und des Evaluationsverfahrens zugänglich gemacht werden, sofern keine anderslautende Genehmigung vom Dualen Partner vorliegt.

Erklärung zur Nutzung Künstlicher Intelligenz

Für die folgende Arbeit wurde die Unterstützung Künstlicher Intelligenz genutzt.
Gemini 2.5 Flash und Gemini 2.5 Pro wurden genutzt, um bei der Literatur Recherche und Fachterminologie Suche zu unterstützen und Rechtschreibung, Grammatik und Ausdruck zu verbessern.
Obwohl diese Hilfsmittel meine Fähigkeiten erweitert und zu meinen Erkenntnissen beigetragen haben, ist es wichtig darauf hinzuweisen das sie inhärente Einschränkungen aufweisen. Ich habe mich bemüht jegliche Unterstützung Künstlicher Intelligenz transparent und verantwortungsvoll zu nutzen.

Diese Erklärung wurde von mir übersetzt, jedoch automatisch von AI Usage Cards generiert. https://ai-cards.org

Abstract

- English -

Abstract1

Abstract

- Deutsch -

Abstract2

Inhaltsverzeichnis

1 Einleitung ... 1
1.1 Motivation ... 1
1.2 Zielsetzung der Arbeit ... 2

2 Methodik ... 3

3 Begriffe, Modell und Anforderungen ... 4
3.1 Begriffsdefinitionen ... 4
3.2 Usage-Evidence-Schema (L0–L2) ... 4
3.3 Anforderungen an den PoC ... 4

4 Grundlagen ... 5
4.1 Software Composition Analysis (SCA) und SBOM ... 5
4.2 Funktionsweise klassischer Scanner (z. B. npm audit) ... 5
4.3 Node.js/TypeScript: Modul-, Build- und Deployment-Grundlagen ... 5
4.4 Call-Graph-Grundlagen für JavaScript/TypeScript ... 5
4.5 Fehlerbilder und Begriffe (FP/FN, Soundness/Precision) ... 5

5 Der aktuelle Forschungsstand ... 6
5.1 Überblick und Einordnung (statisch vs. dynamisch; Evidence-Level) ... 6
5.2 Dynamische Ansätze (Tracing, Coverage, Runtime-Instrumentierung) ... 6
5.3 Statische Ansätze (Reachability, Call-Graphs, modulare Analysen) ... 6
5.4 Vergleich der Ansätze anhand Kriterien (Genauigkeit, Aufwand, K8s-Fit) ... 6
5.5 Ableitungen für den PoC in TypeScript/Node/Kubernetes ... 6

6 Methodenauswahl und PoC-Designentscheidung ... 7
6.1 Bewertungsmatrix und Entscheidungsprozess ... 7
6.2 Gewählte Methode: L2-Reachability via Call-Graph ... 7
6.3 Zielartefakte und Schnittstellen (UsageFactor/Risk-Scoring) ... 7

7 Konzept des Proof of Concept ... 8
7.1 Gesamtpipeline und Architektur ... 8
7.2 L0: Präsenzermittlung (Dependency-Tree/SBOM) ... 8
7.3 L1: Loaded-Heuristiken (optional, falls umgesetzt) ... 8
7.4 L2: Reachability-Analyse (Entry Points, Graph, Mapping) ... 8
7.5 Ergebnisformat und Reporting ... 8

8 Implementierung ... 9
8.1 Tooling-Stack und Projektstruktur ... 9
8.2 Dependency- und Code-Parsing (TS/AST) ... 9
8.3 Call-Graph-Konstruktion und Datenstrukturen ... 9
8.4 Mapping: Vulnerability/Package -> relevante Code-Elemente ... 9
8.5 Beispiel-Durchlauf (Demo-Projekt) ... 9

9 Evaluation ... 10
9.1 Setup und Testkorpus ... 10
9.2 Metriken (Laufzeit, Abdeckung, FP/FN-Stichprobe) ... 10

9.3 Ergebnisse 10
9.4 Vergleich zur Baseline (z. B. npm audit) 10
9.5 Interpretation 10

10 Diskussion und Validität 11
10.1 Limitationen (JS/TS-Dynamik, Bundling, Reflection, DI) 11
10.2 Threats to Validity (intern/extern/konstrukt) 11

11 Fazit und Ausblick 12
11.1 Fazit (Rückbezug auf Forschungsfragen) 12
11.2 Ausblick (z. B. L3, bessere CVE->Symbol-Zuordnung, CI/CD-Integration) 12

Literaturverzeichnis ..... a

Abkürzungsverzeichnis

1 Einleitung

1.1 Motivation

Dependency Scanner wie npm audit und OWASP Dependency-Check sind aus der modernen Softwareentwicklung nicht mehr wegzudenken [1]. Diese Tools basieren primär auf Software Bill of Materials (SBOM) und operieren ausschließlich auf Package-Ebene [2]. Aktuelle Forschung zeigt jedoch fundamentale Limitierungen dieser Ansätze, die zu erheblichen praktischen Problemen führen.

Die Unfähigkeit zu erkennen, ob Code tatsächlich zur Laufzeit ausgeführt wird, führt zu zwei kritischen Problemen. Erstens akkumulieren Projekte obsolete Dependencies, die erhebliche Wartungskosten verursachen [3]. Empirische Analysen zeigen, dass 75,1% aller Maven-Dependencies als „bloated“ klassifiziert werden können, wobei 57% der transitiven Dependencies vollständig ungenutzt sind [4]. Im JavaScript-Ökosystem ist die Situation vergleichbar: 50,7% der Dependencies in CommonJS-Packages sind bloated [5]. Die Entfernung einer einzigen direkten bloated Dependency kann kaskadenartig zur Elimination von bis zu 679 indirekten Dependencies führen [5].

Diese obsoleten Dependencies entstehen durch verschiedene Mechanismen. Transitive Abhängigkeiten werden automatisch in die Dependency-Hierarchie eingefügt, ohne dass Entwickler sich deren Präsenz bewusst sind [3]. Darüber hinaus werden Dependencies während der Entwicklung hinzugefügt und bei Code-Refactorings nicht entfernt [5]. Das Entfernen ist jedoch oft schwierig, weil Entwickler nicht mit ausreichender Sicherheit beurteilen können, ob eine Dependency wirklich entbehrlich ist [3]. Die resultierenden Kosten manifestieren sich in erhöhten Binary-Größen, verlängerten Build-Zeiten, erhöhtem Speicherverbrauch und gesteigertem Sicherheitsrisiko durch eine vergrößerte Angriffsfläche [4], [6]. In containerisierten Deployments wird die Problematik besonders sichtbar, da Images teils eine große Menge veralteter oder verwundbarer JavaScript-Pakete enthalten [7].

Zweitens führt die Fokussierung auf Package-Level-Analysen zu einer hohen Rate an False-Positive-Warnungen. Die zentrale Herausforderung liegt in der fehlenden Granularität: SBOM-basierte Tools können nicht zwischen deployed und nicht-deployed Code differenzieren [8], noch können sie feststellen, ob vulnerable Funktionen tatsächlich vom Anwendungscode aufgerufen werden.

1.2 Zielsetzung der Arbeit

2 Methodik

3 Begriffe, Modell und Anforderungen

3.1 Begriffsdefinitionen

3.2 Usage-Evidence-Schema (L0–L2)

3.3 Anforderungen an den PoC

4 Grundlagen

4.1 Software Composition Analysis (SCA) und SBOM

4.2 Funktionsweise klassischer Scanner (z. B. npm audit)

4.3 Node.js/TypeScript: Modul-, Build- und Deployment-Grundlagen

4.4 Call-Graph-Grundlagen für JavaScript/TypeScript

4.5 Fehlerbilder und Begriffe (FP/FN, Soundness/Precision)

5 Der aktuelle Forschungsstand

5.1 Überblick und Einordnung (statisch vs. dynamisch; Evidence-Level)

5.2 Dynamische Ansätze (Tracing, Coverage, Runtime-Instrumentierung)

5.3 Statische Ansätze (Reachability, Call-Graphs, modulare Analysen)

5.4 Vergleich der Ansätze anhand Kriterien (Genauigkeit, Aufwand, K8s-Fit)

5.5 Ableitungen für den PoC in TypeScript/Node/Kubernetes

6 Methodenauswahl und PoC-Designentscheidung

6.1 Bewertungsmatrix und Entscheidungsprozess

6.2 Gewählte Methode: L2-Reachability via Call-Graph

6.3 Zielartefakte und Schnittstellen (UsageFactor/Risk-Scoring)

7 Konzept des Proof of Concept

7.1 Gesamtpipeline und Architektur

7.2 L0: Präsenzermittlung (Dependency-Tree/SBOM)

7.3 L1: Loaded-Heuristiken (optional, falls umgesetzt)

7.4 L2: Reachability-Analyse (Entry Points, Graph, Mapping)

7.5 Ergebnisformat und Reporting

8 Implementierung

8.1 Tooling-Stack und Projektstruktur

8.2 Dependency- und Code-Parsing (TS/AST)

8.3 Call-Graph-Konstruktion und Datenstrukturen

8.4 Mapping: Vulnerability/Package -> relevante Code-Elemente

8.5 Beispiel-Durchlauf (Demo-Projekt)

9 Evaluation

9.1 Setup und Testkorpus

9.2 Metriken (Laufzeit, Abdeckung, FP/FN-Stichprobe)

9.3 Ergebnisse

9.4 Vergleich zur Baseline (z. B. npm audit)

9.5 Interpretation

10 Diskussion und Validität

10.1 Limitationen (JS/TS-Dynamik, Bundling, Reflection, DI)

10.2 Threats to Validity (intern/extern/konstrukt)

11 Fazit und Ausblick

11.1 Fazit (Rückbezug auf Forschungsfragen)

11.2 Ausblick (z. B. L3, bessere CVE->Symbol-Zuordnung, CI/CD-Integration)

Literaturverzeichnis

[1] S. E. Ponta, H. Plate, und A. Sabetta, „Detection, assessment and mitigation of vulnerabilities in open source dependencies“, *Empirical Software Engineering*, Bd. 25, Nr. 5, S. 3175–3215, 2020, doi: 10.1007/s10664-020-09830-x.

[2] S. E. Ponta, H. Plate, und A. Sabetta, „Beyond Metadata: Code-Centric and Usage-Based Analysis of Known Vulnerabilities in Open-Source Software“, in *2018 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, 2018, S. 449–460. doi: 10.1109/ICSME.2018.00054.

[3] C.-C. Chuang, L. Cruz, R. van Dalen, V. Mikovski, und A. van Deursen, „ Removing dependencies from large software projects: are you really sure? “, in *2022 IEEE 22nd International Working Conference on Source Code Analysis and Manipulation (SCAM)*, Los Alamitos, CA, USA: IEEE Computer Society, Okt. 2022, S. 105–115. doi: 10.1109/SCAM55253.2022.00017.

[4] C. Soto-Valero und others, „A comprehensive study of bloated dependencies in the Maven ecosystem“, *Empirical Software Engineering*, 2021, doi: 10.1007/s10664-020-09914-8.

[5] Y. Liu, D. Tiwari, C. Bogdan, und B. Baudry, „Detecting and removing bloated dependencies in CommonJS packages“, *Journal of Systems and Software*, Bd. 230, S. 112509, Dez. 2025, doi: 10.1016/j.jss.2025.112509.

[6] C. Soto-Valero, T. Durieux, N. Harrand, und B. Baudry, „Coverage-based debloating for java bytecode“, *ACM Transactions on Software Engineering and Methodology*, Bd. 32, Nr. 2, S. 1–34, 2023.

[7] A. Zerouali, V. Cosentino, T. Mens, G. Robles, und J. M. Gonzalez-Barahona, „On the Impact of Outdated and Vulnerable Javascript Packages in Docker Images“, in *2019 IEEE 26th International Conference on Software Analysis, Evolution and Reengineering (SANER)*, 2019, S. 619–623. doi: 10.1109/SANER.2019.8667984.

[8] I. Pashchenko, H. Plate, S. E. Ponta, A. Sabetta, und F. Massacci, „Vuln4Real: A Methodology for Counting Actually Vulnerable Dependencies“, *IEEE Transactions on Software Engineering*, Bd. 48, Nr. 5, S. 1592–1609, 2022, doi: 10.1109/TSE.2020.3025443.